

Introducción al patrón Arquitectónico MVC

MVC, son las siglas de **modelo-vista-controlador** (o en inglés, model-view-controller), que es uno de los tantos **patrones de arquitectura de software**.

Antes de abordar de lleno este patrón, vamos a intentar hacer una introducción a la arquitectura de software, desmembrándola de lo general hacia lo particular, para al fin llegar al detalle, procurando entender exactamente de que se trata, en el contexto adecuado.

Introducción a la Arquitectura de Software

- ¿Qué es la arquitectura de software?

Es necesario aclarar, que no existe una definición única, exacta, abarcadora de “arquitectura de software”. La bibliografía sobre el tema es tan extensa como la cantidad de definiciones que en ella se puede encontrar. Introducir a un concepto simple y sencillo que permita comprender el punto de vista desde el cual.

Concepto

A grandes rasgos, puede decirse que “la Arquitectura de Software es la forma en la que se organizan los componentes de un sistema, interactúan y se relacionan entre sí y con el contexto, aplicando normas y principios de diseño y calidad, que fortalezcan y fomenten la usabilidad a la vez que dejan preparado el sistema, para su propia evolución”.

Tendencias de la Arquitectura de Software

Dos tendencias arquitectónicas, claramente diferenciadas entre sí:

- La **Arquitectura de Software Orientada a Objetos** (como “ingeniería” de sistemas)
- La **Arquitectura Estructurada** (como “desarrollo” de una aplicación)

Otras: la arquitectura basada en patrones y la arquitectura basada en procesos y metodologías.

Características de la Arquitectura de Software:

Atributos de calidad

La **Calidad del Software** puede definirse como **los atributos implícitamente requeridos en un sistema que deben ser satisfechos**. Cuando estos atributos son satisfechos, puede decirse (aunque en forma objetable), que la calidad del software es satisfactoria.

Estos atributos, se gestan desde la arquitectura de software que se emplea, ya sea cumpliendo con aquellos requeridos durante la ejecución del software, como con aquellos que forman parte del proceso de desarrollo de éste.

Atributos de calidad que pueden observarse durante la ejecución del software

1. **Disponibilidad** de uso
2. **Confidencialidad**, puesto que se debe evitar el acceso no autorizado al sistema
3. Cumplimiento de la **Funcionalidad** requerida
4. **Desempeño** del sistema con respecto a factores tales como la capacidad de respuesta
5. **Confiabilidad** dada por la constancia operativa y permanente del sistema
6. **Seguridad externa** evitando la pérdida de información debido a errores del sistema
7. **Seguridad interna** siendo capaz de impedir ataques, usos no autorizados, etc.

Atributos de calidad inherentes al proceso de desarrollo del software

- 8. Capacidad de **Configurabilidad** que el sistema otorga al usuario a fin de realizar ciertos cambios
- 9. **Integrabilidad** de los módulos independientes del sistema
- 10. **Integridad** de la información asociada
- 11. Capacidad de **Interoperar** con otros sistemas (interoperabilidad)
- 12. Capacidad de permitir ser **Modificable** a futuro (modificabilidad)
- 13. Ser fácilmente **Mantenible** (mantenibilidad)
- 14. Capacidad de **Portabilidad**, es decir que pueda ser ejecutado en diversos ambientes tanto de software como de hardware
- 15. Tener una estructura que facilite la **Reusabilidad** de la misma en futuros sistemas
- 16. Mantener un diseño arquitectónico **Escalable** que permita su ampliación (escalabilidad)
- 17. Facilidad de ser **Sometido a Pruebas** que aseguren que el sistema falla cuando es lo que se espera (testeabilidad)

De lo general a lo particular: del estilo arquitectónico al patrón de diseño

Existe una diferencia entre **Estilo Arquitectónico**, **Patrón Arquitectónico** y **Patrón de Diseño**, que debe marcarse a fin de evitar las grandes confusiones que inevitablemente, concluyen en el mal entendimiento y en los resultados poco satisfactorios. Éstos, son los que en definitiva, aportarán “calidad” al sistema resultante. En lo sucesivo, trataremos de establecer la diferencia entre estos tres conceptos, viendo como los mismos, se relacionan entre sí, formando parte de un todo: la arquitectura de software.

Relación y Diferencia

Estilo Arquitectónico, Patrón Arquitectónico y Patrón de Diseño, representan, de lo general a lo particular, los niveles de abstracción que componen la Arquitectura de Software. En este sentido, puede decirse que:

- **El Estilo Arquitectónico** es el encargado de:
 - Describir la *estructura general* de un sistema, *independientemente de otros estilos*
 - Definir los componentes del sistema, su relación e interactividad
 - Ejemplos : *flujo de datos, llamada y retorno, etc.*
- **El Patrón Arquitectónico** es el nivel en el cual la arquitectura de software:
 - Define la *estructura básica* de un sistema, pudiendo estar *relacionado con otros patrones*
 - Representa una *plantilla de construcción* que provee un conjunto de *subsistemas* aportando las *normas para su organización*
 - Ejemplos : *Capas, MVC, Tuberías y Filtros, Pizarra, etc.*
- **El Patrón de Diseño** es el tercer nivel de abstracción de la arquitectura de software, cuya finalidad es la de *precisar en detalle los subsistemas y componentes* de la aplicación
 - Ejemplos : *Proxy, Command, Factory, etc..*

El Patrón Arquitectónico modelo-vista-controlador (MVC)

¿Qué es el patrón MVC?

El patrón MVC es un **patrón de arquitectura de software encargado de separar la lógica de negocio de la interfaz del usuario** y es el más utilizado en aplicaciones Web, ya que facilita la funcionalidad, mantenibilidad y escalabilidad del sistema, de forma simple y sencilla, a la vez que **permite** *“no mezclar lenguajes de programación en el mismo código”*.

MVC divide las aplicaciones en tres niveles de abstracción:

- **Modelo:** representa la lógica de negocios. Es el encargado de acceder de forma directa a los datos actuando como “intermediario” con la base de datos. Lo que en nuestro ejemplo de programación orientada a objetos, serían las clases DBAbstractModel y Usuario.
- **Vista:** es la encargada de mostrar la información al usuario de forma gráfica y “humanamente legible”.
- **Controlador:** es el intermediario entre la vista y el modelo. Es quien controla las interacciones del usuario solicitando los datos al modelo y entregándolos a la vista para que ésta, lo presente al usuario, de forma “humanamente legible”.

¿Cómo funciona el patrón MVC?

El funcionamiento básico del patrón MVC, puede resumirse en:

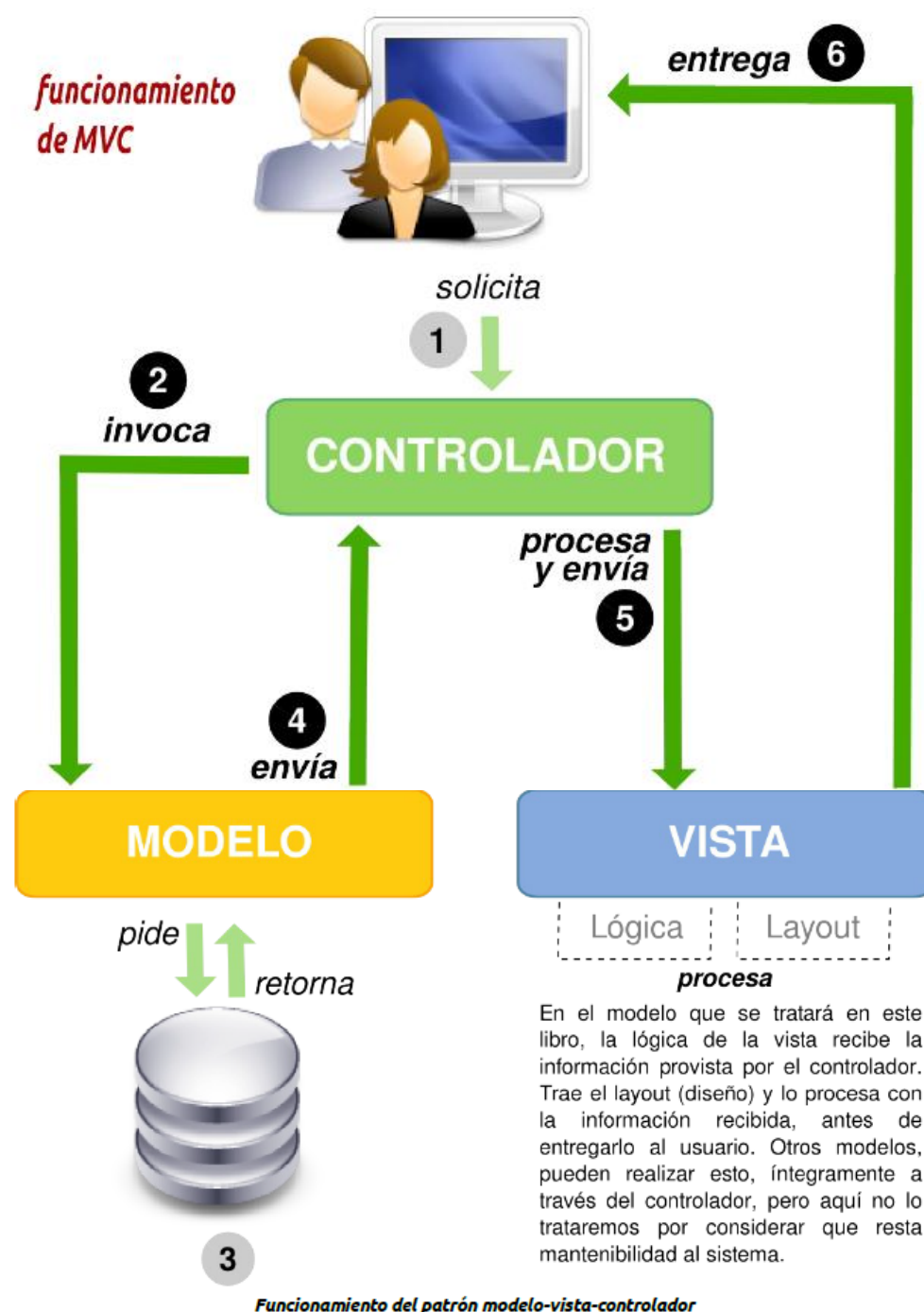
- El **usuario realiza una petición**
- El **controlador captura el evento** (puede hacerlo mediante un manejador de eventos – *handler* -, por ejemplo)
- Hace la **llamada al modelo**/modelos correspondientes (por ejemplo, mediante una llamada de retorno – *callback* -) efectuando las modificaciones pertinentes sobre el modelo
- El **modelo será el encargado de interactuar con la base de datos**, ya sea en forma directa, con una capa de abstracción para ello, un Web Service, etc. Y retornará esta información al controlador

- **El controlador recibe la información y la envía a la vista**
- La vista, procesa esta información pudiendo hacerlo desde el enfoque que veremos en este libro, creando una capa de abstracción para la lógica (quien se encargará de procesar los datos) y otra para el diseño de la interfaz gráfica o GUI.

La lógica de la vista, una vez procesados los datos, los “acomodará” en base al diseño de la GUI - *layout* – y los entregará al usuario de forma “humanamente legible”.

Hasta aquí, hemos hablado de estilos arquitectónicos, nos hemos introducido en uno de los patrones arquitectónicos (MVC) pero, en todo esto ¿en qué momento intervienen los patrones de diseño?

Es aquí donde debemos notar que éstos, intervienen en la forma en la que cada capa (modelo, vista y controlador), “diseña” su estructura. El controlador decidirá (aunque en realidad, nosotros lo haremos) si utilizará un *handler* para manejar los eventos del usuario. En ese caso, estaría optando por un **patrón de diseño**. Si para llamar al modelo, utiliza un *callback*, estaría utilizando otro, y así en lo sucesivo.



El Patrón MVC en PHP

Para que el concepto de modelo-vista-controlador nos quede más claro, vamos a ir analizándolo con ejemplos concretos, en el lenguaje que impulsa este libro: PHP.

El modelo

Con respecto al modelo, no tenemos demasiada novedad. El modelo, no es más que el conjunto de clases con las que hemos trabajado hasta ahora, incluyendo entre ellas, la capa de abstracción de la base de datos.

Interfaces en PHP: un nuevo concepto para crear modelos

En esta etapa, podemos ampliar las referencias de PHP, incorporando un nuevo concepto: **las interfaces**.

El **Manual Oficial de PHP** describe las interfaces de la siguiente forma:

“Las interfaces de objetos permiten crear código con el cual especificar qué métodos deben ser implementados por una clase, sin tener que definir cómo estos métodos son manipulados. Las interfaces son definidas utilizando la palabra clave interface, de la misma forma que con clases estándar, pero sin métodos que tengan su contenido definido. Todos los métodos declarados en una interfaz deben ser public, ya que ésta es la naturaleza de una interfaz.”⁸[...]

Interfaces

Una interface se define utilizando la palabra clave ***interface*** y se la implementa utilizando la palabra clave ***implements***.

En varias ocasiones, las interfaces nos servirán para ampliar los modelos.

Generamos la interface:

```
interface Postre {  
    public function set_ingredientes();  
}
```

Diferencia entre Interfaces y Clases abstractas

Probablemente te preguntes **¿cuál es la diferencia entre una interface y una clase abstracta?**

En principio, existe una **diferencia conceptual** que a niveles “prácticos” tal vez no es del todo clarificadora. Esta diferencia, radica en que ***las clases abstractas, no dejan de ser “clases”, las cuales representan la “abstracción de un objeto” siguiendo un orden de “relación jerárquica” entre ellas:*** Clase B hereda de Clase A y Clase C hereda de Clase B, pero no puede tener herencias múltiples, es decir no puede heredar de más de una clase, ya que ellas, guardan una relación de orden jerárquico entre sí.

```
class A { }
```

```
class B extends class A { } #correcto
```

```
class C extends class B { } #correcto
```

```
class D { } #correcto
```

```
class E extends class A, class D { } # incorrecto!!! no se puede hacer!!!
```

lo correcto sería:

```
class D extends class A { }
```

```
class E extends class D { }
```

Implementamos la interface:

```
class Bizcochuelo implements Postre {  
    var $ingredientes = array();  
    public function set_ingredientes() {  
        $this->ingredientes = array('harina'=>'2 tazas', 'leche'=>'1 taza',  
        'azucar'=>'1 taza', 'huevo'=>1);  
    }  
}
```

Extendemos una clase (que por defecto, implementará la interface):

```
class BizcochueloVainilla extends Bizcochuelo {  
    public function set_ingredientes() {  
        $this->ingredientes['escencia de vainilla'] = 'a gusto';  
    }  
    function __construct() {  
        parent::set_ingredientes();  
        $this->set_ingredientes();  
    }  
}
```

También podemos crear otra clase que implemente la interface:

```
class Alfajor implements Postre {  
    public function set_ingredientes() {  
        $this->ingredientes = array('Tapas de maizena' => 2,  
        'dulce de leche'=>'1 cda. sopera', 'coco rallado'=>'1 cdt. de te');  
    }  
    function __construct() {  
        $this->set_ingredientes();  
    }  
}
```

A diferencia de las clases abstractas, conceptualmente, ***las interfaces son solo un conjunto de métodos característicos de diversas clases***, independientemente de la relación que dichas clases mantengan entre sí. De hecho, una misma clase, puede implementar múltiples interfaces:

```
interface A { }
```

```
interface B { }
```

```
interface C { }
```

```
class MiClase implements A, B, C { }
```


No obstante la diferencia conceptual, **podemos establecer claras diferencias prácticas**, que nos ayudarán a asimilar mejor estos conceptos:

- **Las interfaces no pueden definir propiedades** (de hecho, no poseen propiedades porque a diferencia de las clases abstractas no tienen relación con un objeto, y solo los objetos tienen propiedades)
- **Las interfaces no pueden tener métodos definidos con su algoritmo correspondiente** (solo pueden declarar los métodos pero no pueden indicar el “qué” ni el “cómo” hacerlo, mientras que las clases abstractas, sí pueden)
- **Las interfaces no pueden instanciarse porque no tienen referencias asociadas a objetos**, mientras que las clases abstractas no pueden instanciarse, porque sus objetos son “entes abstractos” que necesitan de una clase no abstracta para definirse con exactitud y poder ser instanciados.
- **Todos los métodos declarados en una interface deben ser públicos**, puesto que la finalidad de una interface, es brindar un “ámbito público de un modelo”. Las clases abstractas, pueden tener métodos abstractos tanto públicos como protegidos (aquellos que deberán ser obligatoriamente redefinidos por las clases que la hereden) y

Lo que NO deben hacer las interfaces

Las interfaces no pueden:

- **Tener el mismo nombre que una clase** (PHP las interpreta como una clase más. Crear una interface con el mismo nombre que una clase, sería interpretado por PHP como una “re-declaración de clase”)
- Diferentes interfaces no pueden **tener nombres de métodos idénticos** si serán implementadas por la misma clase

La vista

Como hemos visto anteriormente, la vista representa la interfaz gráfica del usuario (GUI), es decir, es la encargada de mostrar la información al usuario de manera “humanamente legible”.

A tal fin, dividiremos la vista en dos sub-capas: la GUI propiamente dicha y su lógica.

Vista: la interfaz gráfica

Generalmente, en la práctica, no somos los programadores quienes nos hemos de encargar de la GUI. Es tarea que corresponde a diseñadores Web o gráficos, según aplique.

Como “arquitectos”, debemos tener preparada la estructura de nuestra aplicación, para que diseñadores puedan trabajar libremente en la GUI, sin tener que acceder al código PHP. **Veremos además, como evitar por completo que los diseñadores tengan que implementar código PHP en sus GUI** y que programadores, tengamos que implementar código HTML en nuestras lógicas.

En principio, **la parte gráfica** de las vistas (archivos HTML, imágenes, CSS, etc.), **deberá tener un directorio reservado** solo para ella. De esta forma, obtenemos varias ventajas de trabajar con el patrón MVC:

- **Los archivos estáticos** (HTML, CSS, imágenes e incluso JavaScript) **pueden almacenarse en un servidor independiente** con todos los beneficios que esto conlleva (al igual que sucede con las bases de datos, los archivos estáticos también pueden independizarse)
- **Nos aseguramos de que el código estático** (aquel desarrollado por diseñadores), **no “rompa” el núcleo de la aplicación**
- **Permitimos a los diseñadores trabajar cómodamente** en la libertad de aplicar todos los conocimientos y herramientas inherentes a su área de conocimiento, sin la presión de tener que implementar un lenguaje de programación que desconocen, les es poco familiar o simplemente, la herramienta que utilizan para diseñar, suele dañarles el código no-estático

En segundo lugar, **cada plantilla** – template - o archivo HTML, **deberá hacer referencia “estática” a los datos que se quieran reemplazar dinámicamente.** Para esto, no será necesario implementar código que no sea HTML.

Veremos aquí como hacer referencia a estos datos, y más adelante, veremos como la lógica de la vista se encargará de renderizarlos.

Supongamos que tenemos una plantilla HTML, donde lo que necesitamos reemplazar de manera dinámica, es: **título de la página, keywords y descripción.** La mejor manera de referenciar a estos datos, facilitando la tarea a un diseñador, es haciendo una referencia textual a éstos. Para que a la vez, los programadores logremos diferenciar que allí se incluyen datos dinámicos, una buena práctica, es encerrar esa referencia textual entre llaves.

Veamos un ejemplo:

```
<!DOCTYPE HTML">
<html lang="es">
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
  <title>{Título de la Página}</title>
  <meta name="title" content="{Título de la Página}">
  <meta name="keywords" content="{keywords}">
  <meta name="description" content="{descripción}">
  <link type="text/css" rel="stylesheet" href="css/template.css">
</head>
<body>
<div id="page">
  <div id="cab">
    </a>
  </div>
  <div id="contenido">
    <h1>{Título de la Página}</h1>
  </div>
</div>
</body>
</html>
```

Como podemos ver, en la plantilla HTML solo hay código HTML. Para un diseñador, será una tarea sencilla.

Veremos en el siguiente paso, como la lógica de la vista, se encargará de reemplazar estos datos.

La Lógica de la Vista

Tenemos la plantilla que creó nuestro diseñador. Es hora de reemplazar dinámicamente los datos referenciados. Haremos esto, paso a paso, como si de una receta de cocina se tratara.

Primer paso: crear un diccionario de datos El diccionario de datos es el que contendrá, literalmente hablando, un diccionario, indicando cuales referencias deben ser reemplazadas por cuáles datos, en forma dinámica.

```
$diccionario = array(  
    'Título de la Página'=>'POO y MVC en PHP',  
    'keywords'=>'poo, mvc, php, arquitectura de software',  
    'description'=>'El paradigma de la programación orientada a objetos con el  
    patrón arquitectónico MVC en PHP'  
);
```


Segundo paso: obtener la plantilla HTML

Para traer la plantilla HTML, vamos a utilizar la función **file_get_contents()** de PHP, y almacenar el contenido de ella, en una variable.

```
$template = file_get_contents('/carpeta/template.html');
```

Tercer paso: reemplazar el contenido

En el anteúltimo paso, reemplazaremos los datos, utilizando el bucle **foreach** de PHP para recorrer el diccionario, y la función **str_replace()** de PHP, para reemplazar los mismos.

```
foreach ($diccionario as $clave=>$valor) {  
    $template = str_replace('{' . $clave . '}', $valor, $template);  
}
```

Cuarto paso: mostrar el contenido final al usuario Por último, imprimiremos el resultado obtenido en pantalla.

```
print $template;
```

Y con esto, habremos concluido la lógica de la vista, obteniendo en el navegador del usuario, la siguiente salida (vista de código fuente)

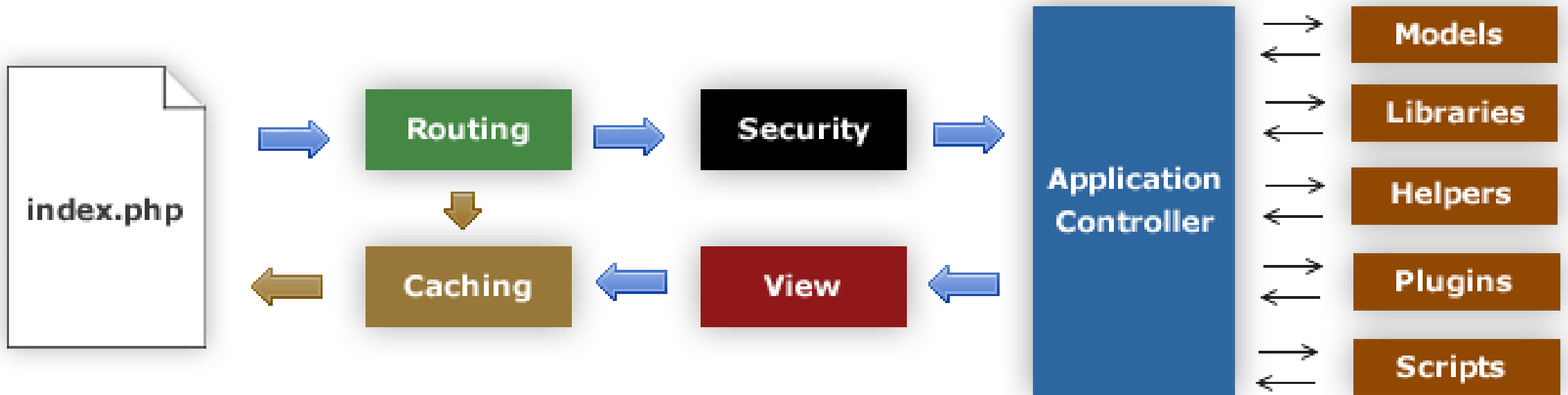
```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html lang="es">

<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
<title>POO y MVC en PHP</title>
<meta name="title" content="POO y MVC en PHP">
<meta name="keywords" content="poo, mvc, php, arquitectura de software">
<meta name="description" content="El paradigma de la programación orientada a
objetos con el
                                patrón arquitectónico MVC en PHP">
<link type="text/css" rel="stylesheet" href="css/template.css">
</head>

<body>
<div id="page">
    <div id="cab">
        </a>
    </div>
    <div id="contenido">
        <h1>POO y MVC en PHP</h1>
    </div>
</div>
</body>

</html>
```

Flujo de una aplicación con CodeIgniter



- 1.The index.php serves as the front controller, initializing the base resources needed to run CodeIgniter.
- 2.The Router examines the HTTP request to determine what should be done with it.
- 3.If a cache file exists, it is sent directly to the browser, bypassing the normal system execution.
- 4.Security. Before the application controller is loaded, the HTTP request and any user submitted data is filtered for security.
- 5.The Controller loads the model, core libraries, helpers, and any other resources needed to process the specific request.
- 6.The finalized View is rendered then sent to the web browser to be seen. If caching is enabled, the view is cached first so that on subsequent requests it can be served.

MVC en CodeIgniter

- The **Model** represents your data structures. Typically your model classes will contain functions that help you retrieve, insert, and update information in your database.
- The **View** is the information that is being presented to a user. A View will normally be a web page, but in CodeIgniter, a view can also be a page fragment like a header or footer. It can also be an RSS page, or any other type of “page”.
- The **Controller** serves as an *intermediary* between the Model, the View, and any other resources needed to process the HTTP request and generate a web page.

En la siguiente clase continuamos con el Controlador y el Modelo

Tarea: Investigar cuáles son las arquitecturas de software más usadas y ejemplifique con sistemas que utilicen estas arquitecturas.

Debe hacer uso de las bases de datos de la biblioteca:

<http://biblio.ecotec.edu.ec/login.php>

Ingresar con usuario y clave de atrium.